

Conceptos y Aplicaciones de Big Data

Definicion

Big data representa una nueva generacion de tecnologias y arquitecturas, diseñadas para extraer **valor** economicamente de **volumenes** muy grandes de una amplia **variedad** de datos, al permitir la captura, el descubrimiento y/o analisis de alta **velocidad**.

Big data no es una tecnologia, es la combinacion de varias para hacer mas facil el tratamiento de datos.

Pueden definirse las 3 V de big data:

- Volumen
- Velocidad
- Variedad

Puede llamarse tambien a la 4ta V **Valor**.

Hadoop

Es una framework que soporta procesamiento de grandes bases de datos en un ambiente distribuido. Ejecuta aplicaciones para el tratamiento de grandes volumenes de datos, incluye un sistema de archivos distribuidos (HDFS), y es tolerante a fallas.

Hadoop fue diseñada para el procesamiento offline de los datos (procesamiento en batch). Funciona con la idea de «escriba una sola vez y lea muchas». No permite lectura aleatoria ni el procesamiento online, ejecutandose localmente donde se encuentren los datos.

DFS (Distributed FileSystem)

Los archivos estan distribuidos, ofreciendo transparencia al usuario permitiendo operar con todos los archivos del cluster a traves del file system distribuido. Un mismo archivo podria estar almacenado en varias computadoras.

Para HDFS, todos los archivos se dividen en bloques del mismo tamaño (64mb o configurable). Los bloques pueden estar fisicamente en cualquier computadora, permitiendo la replica de bloques para optimizacion y recuperio de fallas.

Los procesos se dividen en dos componentes:

- Namenode: maneja el arbol del filesystem y los metadatos de cada archivo y carpeta, conociendo para cada bloque que datanode lo maneja.
- Datanode: son lo que llevan a cabo la lectura y escritura de los bloques en el FS, llevando a cabo la creacion, borrado y replicado de los bloques.

Existe tambien un secondary namemode, realizando tareas auxiliares al namemode.

MapReduce

Es un framework para distribuir tareas en multiples nodos, su espiritu se basa en «escriba una vez y lea muchas». Presenta varias ventajas:

- Paralelizacion y distribucion de tareas automaticas
- Escalable
- Tolerante a fallos
- Monitoreo y capacidad de seguridad
- Flexibilidad de programacion

- Abstraccion al programador

A su vez, es un paradigma de programacion, ya que hay que pensar como resolver un problema sin tener acceso a todos los datos.

Toda tarea MapReduce se reduce en dos etapas:

- Fase Map: los datos de entrada son procesados, uno a uno, y transformados en un conjunto intermedio de datos.
- Fase Reduce: se reúnen los datos intermedios y se reducen a un conjunto de datos, siendo el resultado final de la tarea.

La unidad de trabajo es un Job, que se divide en un map y un reduce. Estos Jobs, son controlados por el JobTracker, donde se almacena el nodo master. Los clientes envían Jobs al JobTracker y este distribuye la tarea usando otros nodos del cluster. Los nodos se conocen como TaskTracker y son responsables de la ejecución de la tarea asignada y reportar el progreso al JobTracker.

El Job es un proceso que se divide en cuatro fases, cada TaskTracker ejecuta la tarea encomendada (map/reduce), mientras que Shuffle y Sort son internas en la ejecución del Job.

Map → Shuffle → Sort → Reduce

Las tareas de map/reduce trabajan con el concepto de $\langle \text{key}, \text{value} \rangle$ y se ejecutan tantos Map como splits existan en el cluster, y tantos Reduce como claves se generaron en las etapas Map.

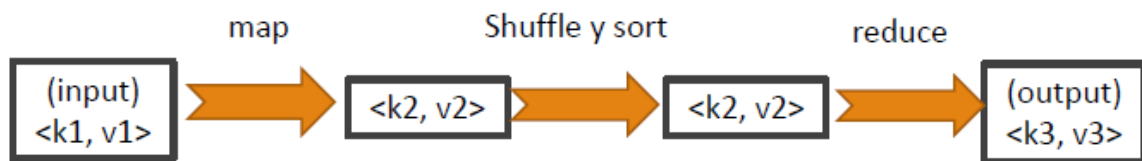


Figura 1: Fases de un Job

Tarea Map

Map recibe datos en forma de pares $(k1, v1)$ y produce una lista de cero, uno o mas pares $(k2, v2)$, donde $k2$ es un identificador para agrupar los datos.

$\langle k1, v1 \rangle \rightarrow \text{list}(\langle k2, v2 \rangle)$

Tarea Reduce

Luego de las tareas intermedias, se ejecuta el reduce, que lee datos en forma de pares $(k2, \text{list}(v2))$ y produce una lista de cero o mas pares $(k3, v3)$.

$\langle k2, \text{list}(v2) \rangle \rightarrow \text{list}(\langle k3, v3 \rangle)$

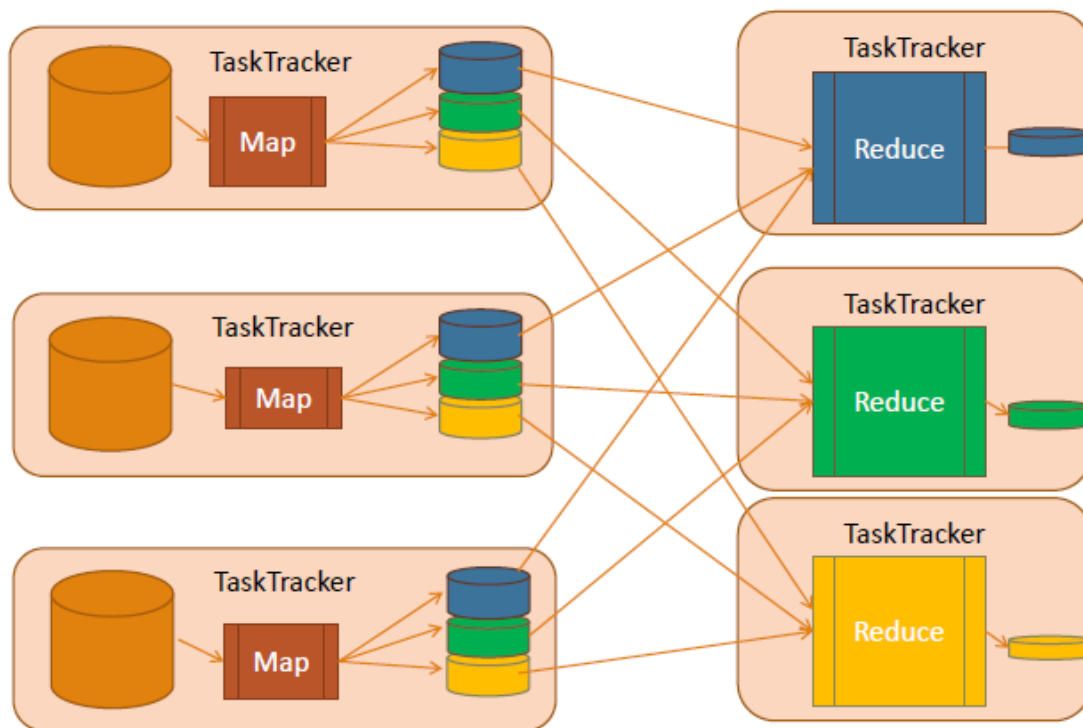


Figura 2: Ejemplo de MapReduce

No hay ninguna forma de saber cuando es la ultima llegada a la funcion, la unica garantia es que, en sucesivas invocaciones, las claves vienen ordenadas. La lista de valores solo puede ser recorrida una vez, siendo inaccesible por indice.

Tarea Combiner

Es una funcion que permite minimizar la cantidad de datos a mover por el cluster al finalizar la etapa de Map y la posterior carga al Reduce. Lo que se busca es que el volumen de datos a guardar en disco sea lo mas chico posible. Se ejecuta en el mismo nodo donde se ejecuto el Map. No hay garantias de cuantas veces se invoca, ya que es una «tarea de optimizacion».

$\langle k2, \text{list}(v2) \rangle \rightarrow \text{list}(\langle k2, v2 \rangle)$

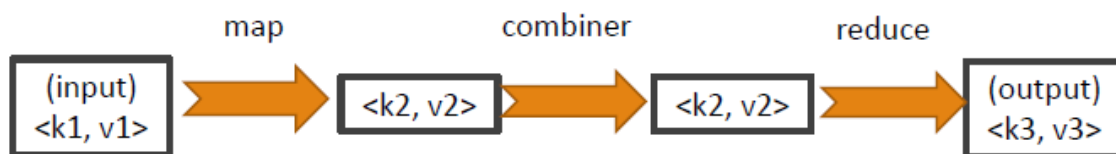


Figura 3: Etapas con combiner

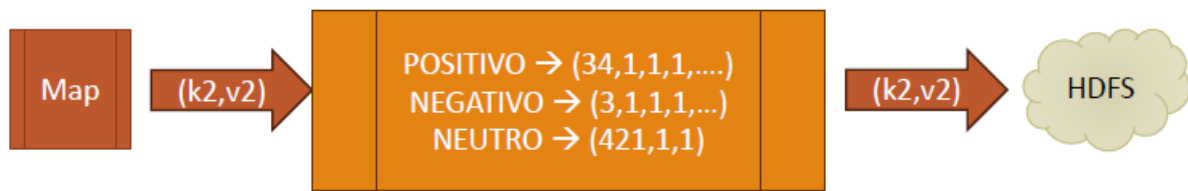


Figura 4: Ejemplo de datos almacenados

Multiples Jobs

A cada Job se le configura un mapper y un reducer (y eventualmente un combiner), y opcionalmente, se pueden ejecutar los mismos para varios Jobs.

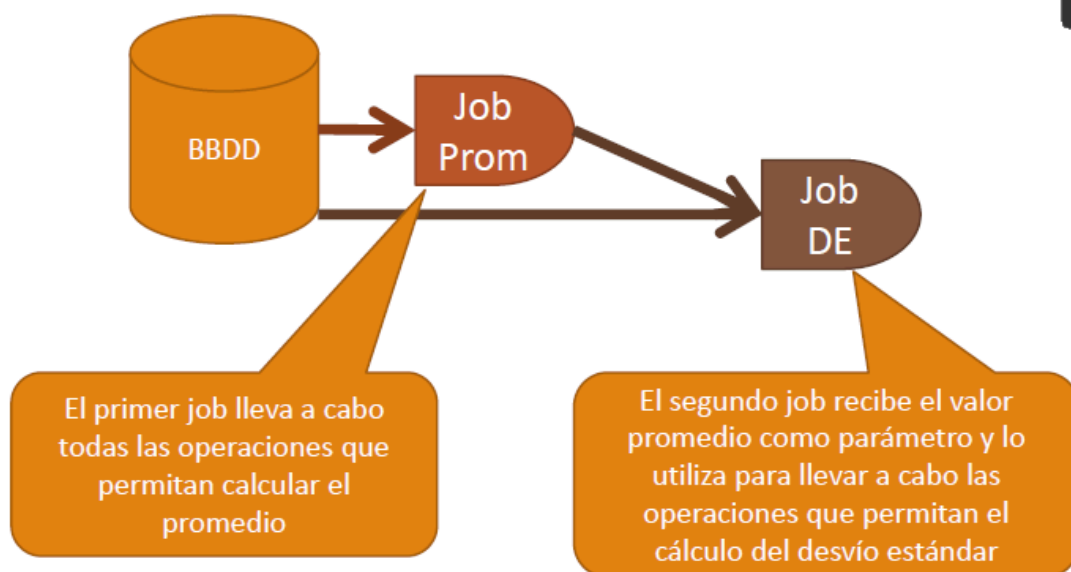


Figura 5: DAG de multiples jobs

Ejemplo de código:

```
job1 = Job(inputDir, tmpDir, fmap1, fred1)
success= job1.waitForCompletion()
job2 = Job(tmpDir, outputDir, fmap2, fred2)
success= job2.waitForCompletion()
```

Operaciones con jobs clásicas

- Group by: usar campo como clave para que el reduce se haga ordenado por ese campo.
- Filter: filtra en la etapa map.
- Projection
- Join
- Shuffle: se encarga de que todas las claves sean procesadas por el mismo reducer.
- Sort: se encarga de que un mismo reducer recibe las tuplas ordenadas por clave.

Spark

Soporta los dos modos de trabajo en Big Data:

- Consultas en grandes volúmenes de datos
- Procesamiento de streaming

Tiene ciertas características:

- Simple
- Veloz
- Soporte

RDDs

Las BD son distribuidas y resilientes, son parte del núcleo de spark y están diseñadas para el almacenamiento de los datos en la ram de manera distribuida en un cluster. Se construyen a partir de la lectura del contenido de un archivo o de una BD, como también de la paralelización de colecciones de datos.

- Eficiencia: paralelización usando varios nodos.
- Tolerante a fallas: se realiza tracking de las operaciones realizadas.
- Son inmutables
- Se dividen en particiones entre los nodos del cluster.

Se pueden realizar dos operaciones básicas, transformaciones (cambia la RDD original a través de un proceso), y acciones (tales como conteos, sumas, calculándose sobre una RDD sin modificarla).

Transformaciones

Toda transformación se aplica sobre una RDD y el resultado es otra RDD.

$\text{Transf}(\text{RDD}_m) \rightarrow \text{RDD}_n$

Transformación Map

Se utiliza para realizar transformaciones sobre tuplas de una RDD, también para la generación de pares clave-valor.

$\text{map}(\text{RDD}_n) \rightarrow \text{RDD}_n$

Es utilizada para crear tuplas de la forma clave-valor.

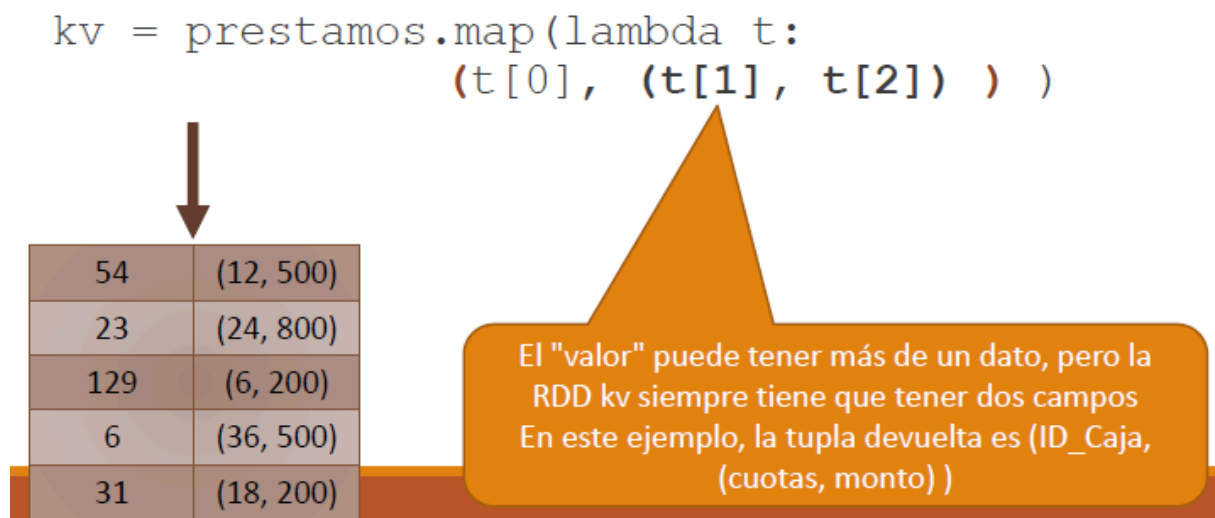


Figura 6: Ejemplo de cómo crear clave-valor con map

Recibe como parametro una funcion, la que tiene implementada la tarea de que hacer sobre cada tupla. Entra tupla, sale tupla.

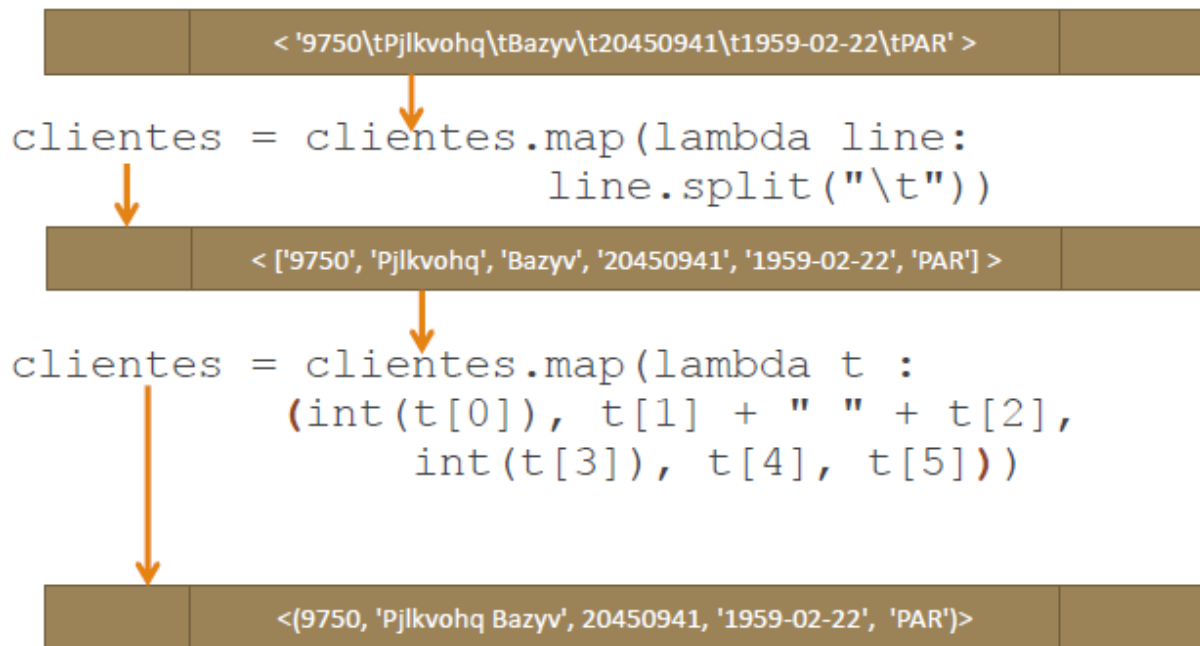


Figura 7: Ejemplo T.Map

Transformacion Filter

Se utiliza para filtrar tuplas en una RDD. Recibe como parametro una tupla y debe devolver un boolean indicando si se pasa o no el filtro pasado como funcion.

$\text{filter}(\text{RDD}_n) \rightarrow \text{RDD}_m, m \leq n$

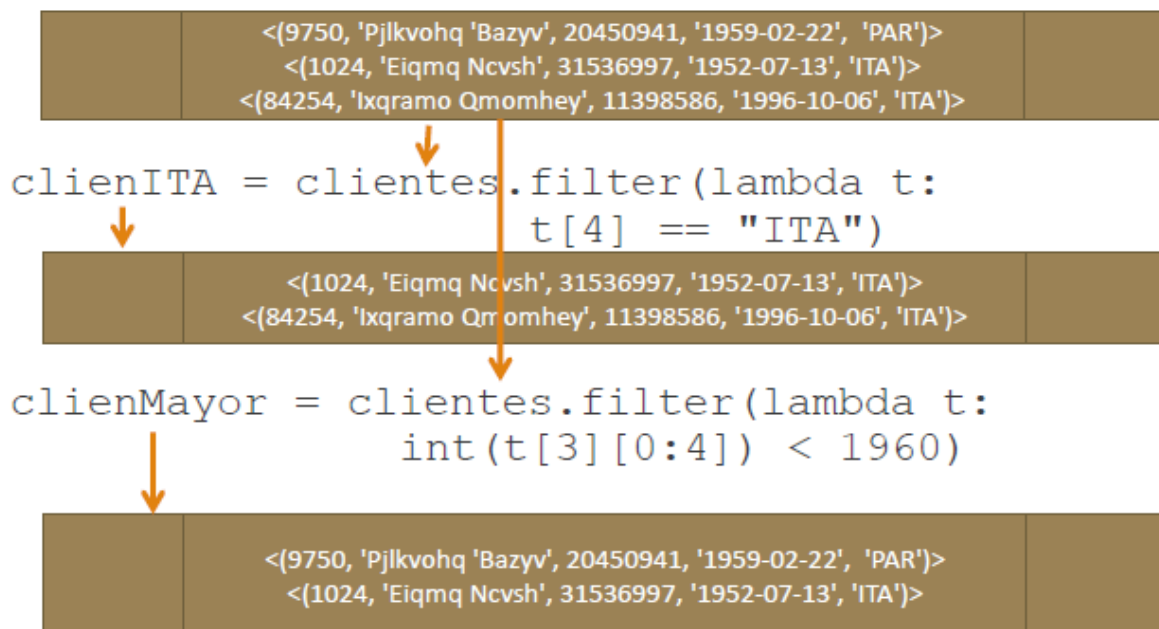


Figura 8: Ejemplo T.Filter

Transformacion Union

Se utiliza para realizar la union entre dos RDDs. Se unen todas las tuplas, sin importar que haya repetidos (no es union de conjuntos).

$\text{union}(\text{RDDn}, \text{RDDm}) \rightarrow \text{RDDn+m}$

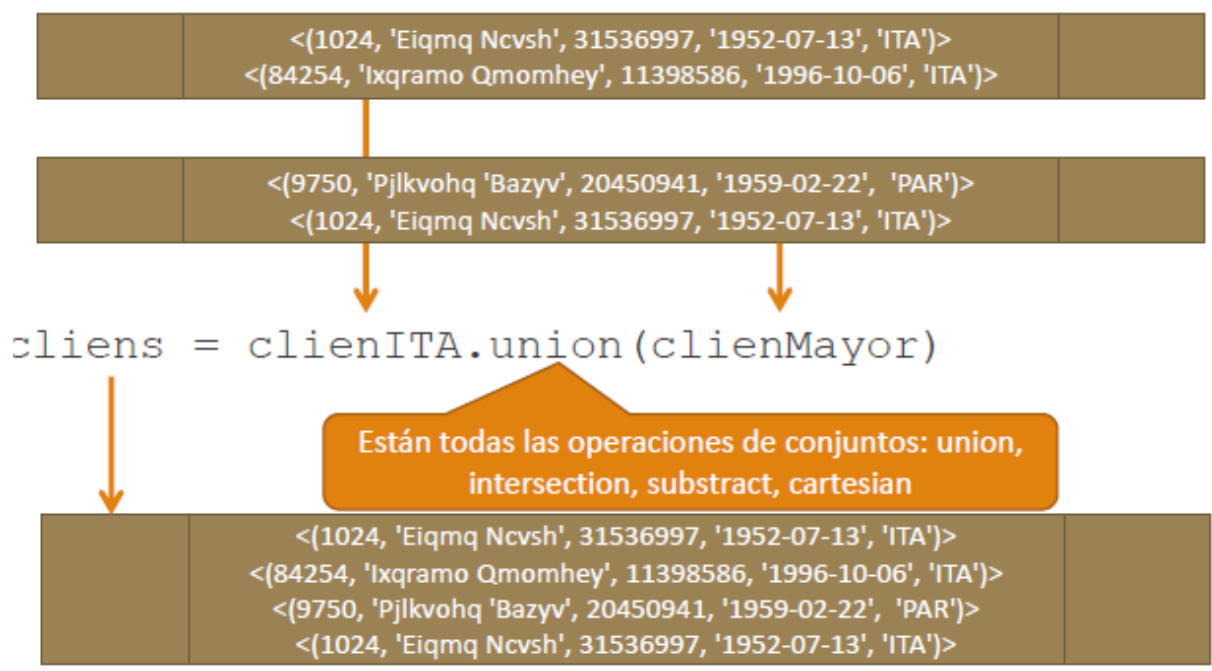


Figura 9: Ejemplo T.Union

Transformacion Distinct

Se utiliza para eliminar tuplas duplicadas dentro de una RDD.

$\text{distinct}(\text{RDDn}) \rightarrow \text{RDDm}, m \leq n$

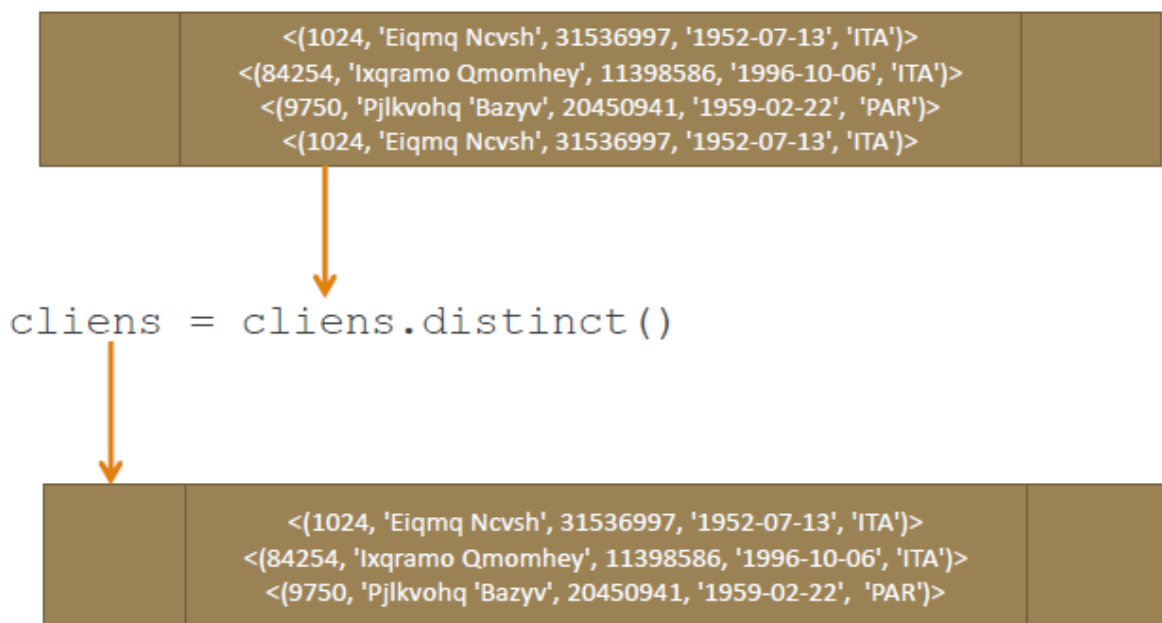


Figura 10: Ejemplo de T.Distinct

Las transformaciones van armando el DAG vinculando las operaciones entre las diferentes RDD que son resultado de tales transformaciones. El DAG (todas las transformaciones) se ejecuta al momento de invocar una acción.



Figura 11: Ejemplo de DAG

Acciones

Toda acción se aplica sobre una RDD y el resultado es un valor que es, o bien devuelto al driver, o bien guardar la RDD en un medio de almacenamiento.

```
cliens.count()
cliens.first()
cliens.take(5)
cliens.takeSample(True, 100)
cliens.top(5)
cliens.collect()
cliens.saveAsTextFile("output")
```

Directorio local o del DFS.

Figura 12: Algunas acciones

Accion Reduce

Se utiliza para reducir/resumir una RDD, haciendose por pares de tuplas. Recibe como parametro una funcion, que recibe dos tuplas y debe devolver otra (del mismo tipo que las que recibio). La comparacion se hace campo a campo.

$\text{freduction}(t1,t2) \rightarrow t3$

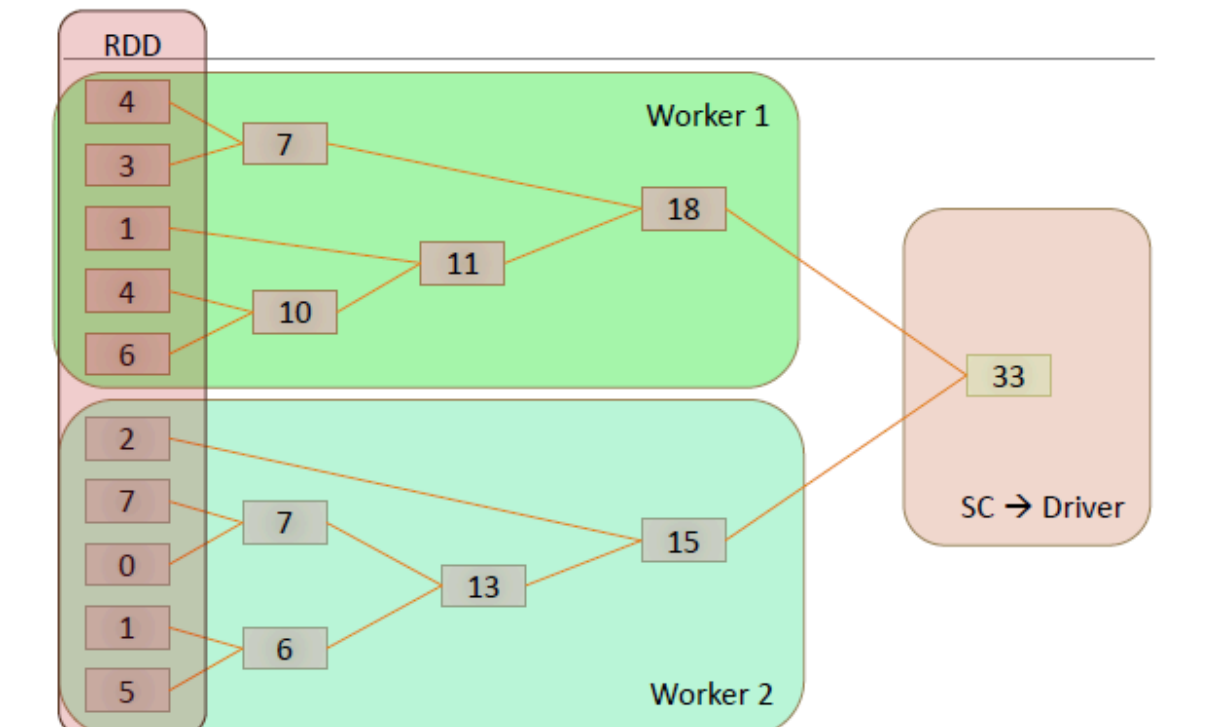


Figura 13: Ejemplo de un Reduce

Persistencia

Cuando una RDD (que depende de muchas otras) se va a utilizar en más de una operación (más de una acción) es conveniente persistirla. Spark posee diferentes tipos de persistencia según las necesidades: en memoria, en disco, ambas.

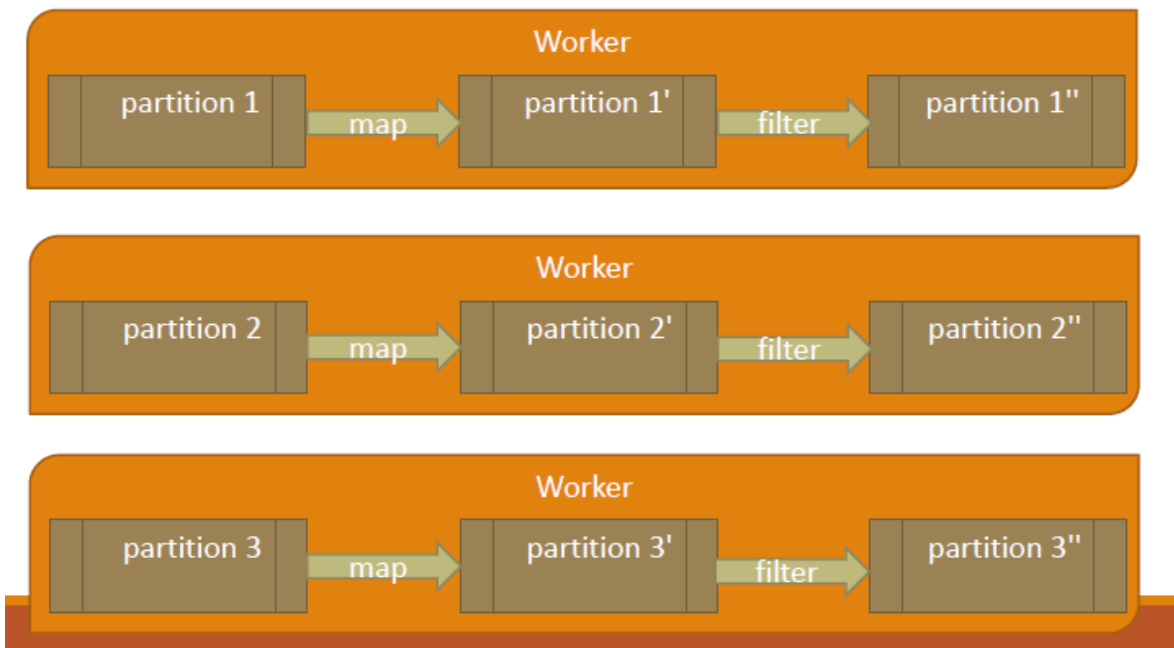
Accion Aggregate

Se utiliza para reducir/resumir una RDD pero cambiando su estructura, tambien se hace por pares de tuplas. Recibe como parametro un valor “cero”, una funcion para operar una tupla original y otra resumida, y otra segunda funcion que sabe operar con dos tuplas resumidas.

Tipos de transformaciones

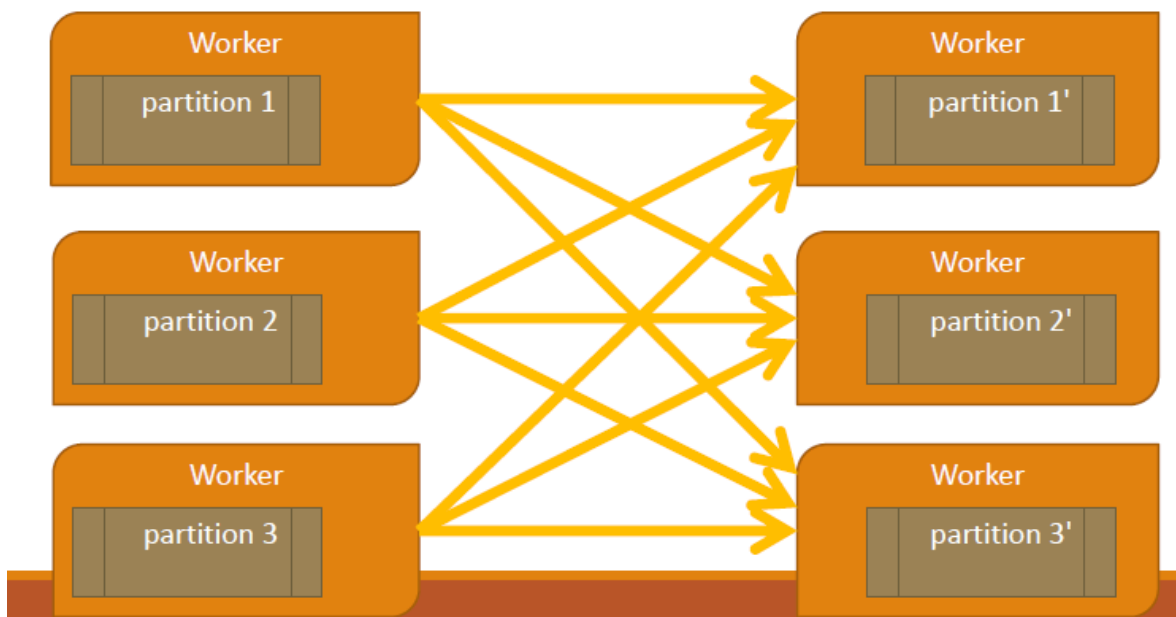
- Narrow (estrechas): pueden ser ejecutadas sobre una particion sin necesidad de transferir datos entre nodos. Se busca maximizar estas transformaciones.

map, filter, ...



- Wide (amplias): necesitan transferir datos entre nodos para poder realizar la transformacion. Se busca minimizar estas transformaciones.

groupBy, join, ...



Acumuladores

Son variables especiales de Spark que permiten llevar a cabo un acumulado global entre todos los nodos del cluster. Cada nodo lleva a cabo su propio acumulado. Cuando el driver solicita el valor del acumulador, se realiza el merge entre todos los acumulados parciales de todos los workers. Su propósito es el de debugging.

Parallelize

SparkContext tiene un método parallelize que se utiliza para generar una RDD desde una colección de valores (listas, tuplas, diccionarios, conjuntos). Cada elemento de la lista se convierte en una tupla dentro de la nueva RDD.

Variables Broadcast

Una variable broadcast se crea a partir del SparkContext. Se envía de manera eficiente a todos los nodos del cluster. Se puede almacenar cualquier estructura: entero, string, lista, diccionario, etc. Al valor almacenado en una variable broadcast se accede mediante la propiedad value.

SQL

Es una interface que permite usar Spark con datos estructurados, es decir con datos que tienen un esquema. Permite el uso del lenguaje SQL para realizar tareas de selección, filtrado, agrupación y joins.

Spark SQL trabaja con una abstracción de las RDD, los DataFrames. Un DataFrames una RDD con esquema, es decir tuplasde datos con nombres y tipo de datos de cada campo. Estos DataFrames son distribuidos.

Stream Processing

El flujo de datos es continuo, recolectados en tiempo real, sin entrenarse para el modelo, usandose y descartandose sin almacenarse.

Un algoritmo de streaming debe cuidar tres aspectos:

- Velocidad
- Memoria
- Eficacia